Regression Analysis, Resampling Methods, and Gradient Descent: A Numerical Study on Runge's Function

Daniel Junior Lifenya Fondo
Department of Physics, University of Oslo, N-0316 Oslo, Norway
(Dated: October 26, 2026)

We study polynomial regression on Runge's function $f(x) = 1/(1+25x^2)$ with additive Gaussian noise ($\sigma = 0.1$, $n = 100$), using Ordinary Least Squares (OLS), Ridge, and Lasso regression. We apply bootstrap resampling to decompose test error into bias, variance, and noise, and k-fold cross-validation ($k = 5, 10$) for model selection. Analytical gradients are benchmarked against automatic differentiation (JAX), and optimization performance is evaluated across GD, Momentum, AdaGrad, RMSprop, Adam, and SGD. Closed-form SVD solvers and gradient descent agree to machine precision, while Ridge regularization successfully suppresses boundary oscillations caused by the Runge phenomenon.

I. INTRODUCTION

The goal of this project is to analyze Runge's function by fitting noisy data with polynomials—establishing machine learning foundations while encapsulating statistical properties as well as optimization protocols. Runge's function is an especially good test-function because it allows us to discriminate different behaviors depending on intervals. This allows us to establish the notions of test and training error, and explore model usage depending on the interval at hand. The function itself is smooth, but has strong curvature near the boundaries. Attempting to fit it with high-degree polynomials causes the so-called *Runge phenomenon*. This is a classic example used to motivate regularization in [**?**], Chapter 3.

The main part of this work explores theory and method, where we journey through polynomial regression, OLS, Ridge, Lasso, the bias-variance tradeoff, resampling methods, and gradient-based optimization, finalized with a model-selection comparison across all methods.

II. THEORY AND METHODS

A. Data Generation and Model Setup

Data are generated from Runge's function with additive Gaussian noise:

$$y_i = \frac{1}{1+25x_i^2} + \varepsilon_i, \quad x_i \sim \mathcal{U}(-1,1), \quad \varepsilon_i \sim \mathcal{N}(0,\sigma^2), \tag{1}$$

with $n = 100$ samples and noise level $\sigma = 0.1$. We fit degree-d polynomials $\tilde{y}(x;\boldsymbol{\theta}) = \sum_{j=0}^{d} \theta_j x^j = \boldsymbol{X}\boldsymbol{\theta}$. To handle severe ill-conditioning at high d, non-intercept columns of the design matrix $\boldsymbol{X}$ are standardized to zero mean and unit variance using training-set statistics only. The target $\boldsymbol{y}$ is centered, decoupling the intercept from ℓ_2 regularization.

B. Estimators and Closed-Form Solvers

For Ordinary Least Squares (OLS), minimizing $C(\boldsymbol{\theta}) = \frac{1}{n}\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}\|^2$ via normal equations $(\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\boldsymbol{y}$ fails at high polynomial degrees as $\kappa(\boldsymbol{X}^T\boldsymbol{X}) \to 10^{13}$. We employ Singular Value Decomposition (SVD) $\boldsymbol{X} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^T$, yielding:

$$\boldsymbol{\theta}^*_{\text{OLS}} = \sum_{i=1}^{p} \frac{\boldsymbol{u}_i^T\boldsymbol{y}}{\sigma_i}\boldsymbol{v}_i, \quad \boldsymbol{\theta}^*_{\text{Ridge}} = \boldsymbol{V}\,\text{diag}\left(\frac{\sigma_i}{\sigma_i^2+\lambda}\right)\boldsymbol{U}^T\boldsymbol{y}. \tag{2}$$

Ridge suppresses singular values $\sigma_i^2 \ll \lambda$, directly dampening the high-frequency polynomial modes responsible for boundary oscillations.

C. Resampling & Gradient Optimization

Test MSE is decomposed into $\text{Bias}^2 + \text{Variance} + \sigma^2$ via $B = 100$ bootstrap draws on the training split. Model selection is verified independently using k-fold cross-validation ($k = 5, 10$). For gradient-based optimization, gradients $\nabla C(\boldsymbol{\theta}) = \frac{2}{n}\boldsymbol{X}^T(\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y})$ are verified against reverse-mode automatic differentiation in JAX. Convergence is bounded by learning rate $\eta < \frac{2}{\lambda_{\max}}(\boldsymbol{H})$, where $\boldsymbol{H} = \frac{2}{n}\boldsymbol{X}^T\boldsymbol{X}$. We model a scalar target y as a polynomial function of a single input $x \in [-1, 1]$:

$$\tilde{y}(x;\boldsymbol{\theta}) = \sum_{j=0}^{d} \theta_j x^j, \tag{3}$$

where d is the polynomial degree and $\boldsymbol{\theta} = (\theta_0, \ldots, \theta_d)^T$ are the coefficients. Given n observations (x_i, y_i) we assemble the design matrix

$$\boldsymbol{X} = \begin{pmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^d \\ 1 & x_2 & x_2^2 & \cdots & x_2^d \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^d \end{pmatrix} \in \mathbb{R}^{n\times(d+1)}. \tag{4}$$

The prediction for all n samples is $\tilde{\boldsymbol{y}} = \boldsymbol{X}\boldsymbol{\theta}$. The columns of $\boldsymbol{X}$ have widely different scales: x^j has column

$\mathcal{O}(\sqrt{n/(2j+1)})$. For $j = 15$, this is only a factor ~ 3 smaller than x^1-modest in absolute terms, but enough to inflate $\kappa\left(\mathbf{X}^T\mathbf{X}\right)$ to $\mathcal{O}(10^4)$ at degree 15, which is where the numerical problems begin. As we will see, this disparity makes the unscaled problem ill-conditioned, motivating the standardization step described in Sec. II M.

D. Ordinary Least Squares

The ordinary least squares (OLS) estimator minimises the mean squared error between the prediction and the target:

$$C(\boldsymbol{\theta}) = \frac{1}{n}\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}\|^2. \tag{5}$$

Setting the gradient to zero gives the normal equations

$$\boldsymbol{X}^T\boldsymbol{X}\,\boldsymbol{\theta}^* = \boldsymbol{X}^T\boldsymbol{y}, \tag{6}$$

with formal solution:

$$\boldsymbol{\theta}^* = (\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\boldsymbol{y} \tag{7}$$

Numerically, we solve this via the singular value decomposition (SVD) $\boldsymbol{X} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^T$:

$$\boldsymbol{\theta}^* = \boldsymbol{V}\boldsymbol{\Sigma}^{-1}\boldsymbol{U}^T\boldsymbol{y} = \sum_{i=1}^{p}\frac{\boldsymbol{u}_i^T\boldsymbol{y}}{\sigma_i}\boldsymbol{v}_i, \tag{8}$$

where $p = d+1$ is the number of columns and $p \leq n$. The SVD form is numerically preferable to forming $\boldsymbol{X}^T\boldsymbol{X}$ explicitly: it avoids squaring the condition number, and singular values σ_i below a tolerance $r\sigma_{\max}$ are set to zero, producing the minimum-norm least-squares solution when $\boldsymbol{X}$ is rank-deficient[1].

Numerically this matters only in the numerical null space of $\mathbf{X}$: the two solvers produce nearly identical predictions on the training range, but the formal solve's coefficients contain enormous components in the near-null directions, which cause catastrophic divergence under extrapolation (Fig.1). Geometrically, $\boldsymbol{X}\boldsymbol{\theta}^*$ is the orthogonal projection of $\boldsymbol{y}$ onto the column space of $\boldsymbol{X}$.

E. Ridge Regression and SVD Shrinkage

Ridge regression adds an ℓ_2 penalty on the coefficients:

$$C_\lambda(\boldsymbol{\theta}) = \frac{1}{n}\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}\|^2 + \lambda\|\boldsymbol{\theta}\|^2. \tag{9}$$

[1] In the context of this work we regard this as the non-invertible property of a matrix $\mathcal{A} = \mathbf{X}^T\mathbf{X}$ or the dimensional mismatch of having more features (larger columns vectors) than observations/samples (row vectors)–including extra non-sensual features of observations

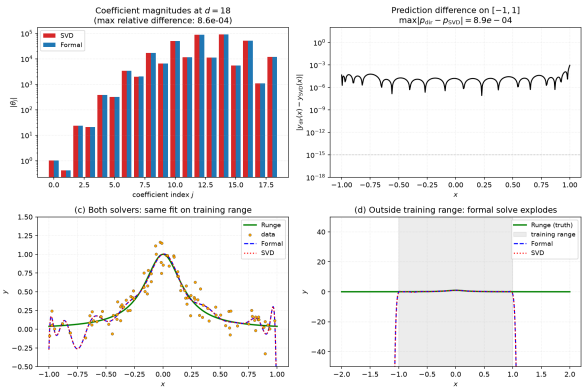


FIG. 1. Numerical comparison of the two OLS solvers at polynomial degree $d = 18$, where $\kappa(\boldsymbol{X}^T\boldsymbol{X}) \sim 10^{13}$. (a) The coefficient vectors produced by the formal normal-equation solve and by the SVD. The high-order coefficients of the formal solution are many orders of magnitude larger than those of the SVD solution. (b) The difference between the two predictions on the training interval is at machine precision ($\sim 10^{-15}$), and (c) the fitted curves are visually indistinguishable: the coefficient discrepancy lives in the numerical null space of $\boldsymbol{X}$. (d) Outside the training range, however, the formal solution diverges catastrophically, while the SVD solution remains bounded. The SVD's tolerance-based truncation of small singular values is what prevents this blow-up: it produces the minimum-norm $\boldsymbol{\theta}$ whose predictions are stable both inside and outside the training range.

The closed-form solution is

$$\boldsymbol{\theta}^*_\lambda = (\boldsymbol{X}^T\boldsymbol{X} + \lambda\boldsymbol{I})^{-1}\boldsymbol{X}^T\boldsymbol{y}. \tag{10}$$

Substituting the SVD of $\boldsymbol{X}$ into Eq. (10) and using $\boldsymbol{U}^T\boldsymbol{U} = \boldsymbol{I}$ and $\boldsymbol{V}^T\boldsymbol{V} = \boldsymbol{I}$, the Ridge solution can be written as:

$$\boldsymbol{\theta}^*_\lambda = \boldsymbol{V}\,\mathrm{diag}\left(\frac{\sigma_i}{\sigma_i^2 + \lambda}\right)\boldsymbol{U}^T\boldsymbol{y}. \tag{11}$$

Comparing Eq. (11) with Eq. (8), we see that Ridge replaces the factor $1/\sigma_i$ with $\sigma_i/(\sigma_i^2 + \lambda)$:

$$\frac{1}{\sigma_i} \longrightarrow \frac{\sigma_i}{\sigma_i^2 + \lambda}. \tag{12}$$

This is the *shrinkage of the singular-value modes*. For directions with $\sigma_i^2 \gg \lambda$, the factor approaches $1/\sigma_i$ and the solution coincides with OLS. For directions with $\sigma_i^2 \ll \lambda$, the factor becomes $\sigma_i/\lambda \to 0$ and those directions are suppressed. Ridge therefore attacks the *variance* of the estimator: the small-σ_i directions, which OLS amplifies by a factor $1/\sigma_i$, are instead muted. In polynomial regression these small-σ_i directions are precisely the high-degree polynomial modes that produce the Runge phenomenon.

F. Lasso Regression

Lasso replaces the ℓ_2 penalty with an ℓ_1 penalty:

$$C_\lambda(\boldsymbol{\theta}) = \frac{1}{n}\|\boldsymbol{y}-\boldsymbol{X}\boldsymbol{\theta}\|^2+\lambda\|\boldsymbol{\theta}\|_1, \qquad \|\boldsymbol{\theta}\|_1 = \sum_j |\theta_j|. \quad (13)$$

The ℓ_1 norm is not differentiable at $\theta_j = 0$. Formally, the subdifferential at the origin is $[-1, 1]$, and the optimization condition at a candidate solution $\boldsymbol{\theta}^*$ is that the gradient of the smooth part be in $-\lambda\,\partial\|\boldsymbol{\theta}^*\|_1$. In practice, exact sparsity requires the use of proximal methods (soft-thresholding) or coordinate descent, which are outside the scope of this project. We instead use the automatic-differentiation subgradient, which returns the value 0 at $\theta_j = 0$. This is a valid subgradient of $|\theta|$ at that point, but does not enforce sparsity: coefficients approach zero asymptotically but do not become exactly zero. This distinction is important for interpreting the Lasso results in Sec. III G.

G. The Bias-Variance Decomposition

Consider data generated from a noisy process

$$y = f(x) + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, \sigma^2). \quad (14)$$

Let $\tilde{y} = \tilde{y}(x; \mathcal{L})$ be a model fitted to a training set $\mathcal{L}$, and let $\mathbb{E}_\mathcal{L}$ denote the expectation over the randomness of $\mathcal{L}$. The expected test error at a point x satisfies

$$\begin{aligned}\mathbb{E}_{\mathcal{L},\varepsilon}\big[(y-\tilde{y})^2\big] &= \mathbb{E}_{\mathcal{L},\varepsilon}\Big[\big(f+\varepsilon-\tilde{y}\big)^2\Big] \\ &= \mathbb{E}_\mathcal{L}\big[(f-\tilde{y})^2\big] + 2\mathbb{E}[\varepsilon]\,\mathbb{E}_\mathcal{L}[f-\tilde{y}] + \mathbb{E}[\varepsilon^2] \\ &= \mathbb{E}_\mathcal{L}\big[(f-\tilde{y})^2\big] + \sigma^2, \end{aligned} \quad (15)$$

where we have used $\mathbb{E}[\varepsilon] = 0$ and independence of ε and $\mathcal{L}$. Adding and subtracting $\mathbb{E}_\mathcal{L}[\tilde{y}]$ inside the first term:

$$f - \tilde{y} = \big(f - \mathbb{E}_\mathcal{L}[\tilde{y}]\big) + \big(\mathbb{E}_\mathcal{L}[\tilde{y}] - \tilde{y}\big). \quad (16)$$

Squaring and taking the expectation, the cross-term vanishes because $\mathbb{E}_\mathcal{L}[\mathbb{E}_\mathcal{L}[\tilde{y}] - \tilde{y}] = 0$. We obtain

$$\mathbb{E}\big[(y-\tilde{y})^2\big] = \underbrace{\big(f-\mathbb{E}_\mathcal{L}[\tilde{y}]\big)^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}_\mathcal{L}\big[(\tilde{y}-\mathbb{E}_\mathcal{L}[\tilde{y}])^2\big]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{noise}}. \quad (17)$$

The three terms have the following interpretations. *Bias* measures how far the average prediction over training sets is from the true function: a low-degree polynomial cannot bend enough and has high bias. *Variance* measures how much the prediction fluctuates as the training set changes: a high-degree polynomial fits the noise and has high variance. The *noise* term is irreducible and is a floor on the expected test error.

In practice $f(x)$ is unknown, and we replace it with the noisy target y when measuring bias from data:

$$\mathbb{E}\big[(y-\mathbb{E}[\tilde{y}])^2\big] = \text{Bias}^2 + \sigma^2. \quad (18)$$

Our measured bias therefore already absorbs the noise variance, and the sum $\widehat{\text{Bias}^2} + \widehat{\text{Var}}$ directly estimates the expected test MSE. This is important when interpreting the numerical bias curves in Sec. III C.

H. Bootstrap Resampling

The bootstrap estimates the expectation $\mathbb{E}_\mathcal{L}$ appearing in Eq. (17) by resampling from the training set. Given a training set of size n_{tr}, we draw B bootstrap samples, each obtained by sampling n_{tr} indices uniformly at random with replacement. For each bootstrap sample $b = 1, \ldots, B$ we fit a model, obtaining a coefficient vector $\boldsymbol{\theta}^{(b)}$, and predict on a fixed test set $\{\boldsymbol{x}_i^{\text{te}}\}_{i=1}^{n_{\text{te}}}$. Let $\tilde{y}_i^{(b)}$ denote the prediction on test point i in bootstrap iteration b. We then estimate

$$\widehat{\text{Bias}^2}(x_i) = \big(y_i^{\text{te}} - \bar{\tilde{y}}_i\big)^2, \quad (19)$$

$$\widehat{\text{Var}}(x_i) = \frac{1}{B}\sum_{b=1}^{B}\big(\tilde{y}_i^{(b)} - \bar{\tilde{y}}_i\big)^2, \quad (20)$$

where $\bar{\tilde{y}}_i = B^{-1}\sum_b \tilde{y}_i^{(b)}$. The quantities reported in the results section are the averages of these expressions over the test set. The preprocessing (standardization of $\boldsymbol{X}$, centering of $\boldsymbol{y}$) is fit once on the full training set and held fixed across bootstrap iterations: the bootstrap is simulating resampled training data, not resampled preprocessing.

I. Cross-Validation

k-fold cross-validation partitions the data into k disjoint subsets (folds) of size $\lceil n/k \rceil$. For each fold $j \in \{1, \ldots, k\}$, we train on the remaining $k-1$ folds and validate on fold j, obtaining a mean squared error MSE_j. The cross-validation score is

$$\text{CV}_k = \frac{1}{k}\sum_{j=1}^{k}\text{MSE}_j. \quad (21)$$

Every observation is validated exactly once, and every observation is used in the training set of $k-1$ folds, so no data are wasted. Because each fold is unseen during its own training, the CV score is an unbiased estimate of test error under the i.i.d. assumption.

Preprocessing that depends on the data (standardization of $\boldsymbol{X}$, centering of $\boldsymbol{y}$) is performed *inside* the cross-validation loop. The scaler is fit on the training folds and applied to the validation fold. Fitting it once on all data before splitting would leak information from the validation fold into the training fold and produce an optimistically biased CV score.

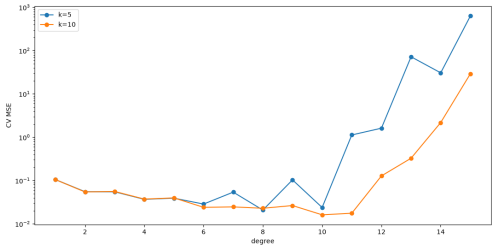


FIG. 2. CV and bootstrap agree on optimal complexity within one degree

J. Gradient Descent and Adaptive Optimizers

The closed-form solutions in Secs. II D and II E are available only for quadratic costs. For general differentiable costs, the standard approach is gradient descent. Starting from an initial guess $\boldsymbol{\theta}_0$, we iterate

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta \nabla C(\boldsymbol{\theta}_k), \tag{22}$$

where η is the *learning rate*. For OLS and Ridge, the gradients are

$$\nabla C_{\text{OLS}}(\boldsymbol{\theta}) = \frac{2}{n}\boldsymbol{X}^T(\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y}), \tag{23}$$

$$\nabla C_{\text{Ridge}}(\boldsymbol{\theta}) = \frac{2}{n}\boldsymbol{X}^T(\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y}) + 2\lambda\boldsymbol{\theta}. \tag{24}$$

Because the cost is quadratic, the Hessian is $\boldsymbol{H} = \frac{2}{n}\boldsymbol{X}^T\boldsymbol{X}$ (plus $2\lambda\boldsymbol{I}$ for Ridge), and the iteration in Eq. (22) converges if and only if

$$\eta < \frac{2}{\lambda_{\max}(\boldsymbol{H})}. \tag{25}$$

This is the standard convergence criterion for gradient descent on a quadratic form and directly ties the safe learning rate to the largest eigenvalue of the Hessian.

Four adaptive variants are considered:

a. Momentum. Maintains a running average of past gradients:

$$\boldsymbol{v}_{k+1} = \beta\boldsymbol{v}_k + (1-\beta)\nabla C(\boldsymbol{\theta}_k), \tag{26}$$

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta\boldsymbol{v}_{k+1}. \tag{27}$$

Momentum smooths the trajectory and damps oscillations along directions of alternating gradient sign, effectively accelerating convergence on ill-conditioned problems.

b. AdaGrad. Accumulates squared gradients and scales the learning rate per-coordinate:

$$\boldsymbol{G}_k = \boldsymbol{G}_{k-1} + \nabla C(\boldsymbol{\theta}_k)^2, \tag{28}$$

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \frac{\eta}{\sqrt{\boldsymbol{G}_k} + \epsilon}\nabla C(\boldsymbol{\theta}_k). \tag{29}$$

AdaGrad provides smaller effective steps for parameters that have moved a lot; its accumulated sum causes the effective learning rate to decay monotonically, which can stall on long runs.

c. RMSprop. Replaces the sum in AdaGrad with an exponential moving average:

$$\boldsymbol{s}_k = \beta\boldsymbol{s}_{k-1} + (1-\beta)\nabla C(\boldsymbol{\theta}_k)^2, \tag{30}$$

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \frac{\eta}{\sqrt{\boldsymbol{s}_k} + \epsilon}\nabla C(\boldsymbol{\theta}_k). \tag{31}$$

d. Adam. Combines momentum and RMSprop with bias correction:

$$\boldsymbol{m}_k = \beta_1\boldsymbol{m}_{k-1} + (1-\beta_1)\nabla C(\boldsymbol{\theta}_k), \tag{32}$$

$$\boldsymbol{v}_k = \beta_2\boldsymbol{v}_{k-1} + (1-\beta_2)\nabla C(\boldsymbol{\theta}_k)^2, \tag{33}$$

$$\hat{\boldsymbol{m}}_k = \frac{\boldsymbol{m}_k}{1-\beta_1^k}, \qquad \hat{\boldsymbol{v}}_k = \frac{\boldsymbol{v}_k}{1-\beta_2^k}, \tag{34}$$

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \frac{\eta\,\hat{\boldsymbol{m}}_k}{\sqrt{\hat{\boldsymbol{v}}_k} + \epsilon}. \tag{35}$$

The bias correction terms $1/(1-\beta_i^k)$ compensate for the zero-initialization of the moment estimates, which would otherwise produce systematically small effective steps in early iterations.

K. Stochastic Gradient Descent

Stochastic gradient descent (SGD) replaces the full-batch gradient with a mini-batch estimate. At each step we sample a subset $\mathcal{B} \subset \{1, \ldots, n\}$ of size B and use

$$\widehat{\nabla C}(\boldsymbol{\theta}) = \frac{1}{B}\sum_{i\in\mathcal{B}}\nabla\ell_i(\boldsymbol{\theta}). \tag{36}$$

Because the mini-batch is drawn uniformly at random, $\widehat{\nabla C}$ is an unbiased estimator of ∇C, with variance proportional to $1/B$. The parameter update follows any of the optimizers above. One full pass through the training set is called an *epoch*.

Small batches give noisier but more frequent updates; large batches give smoother but more expensive updates. The optimal batch size balances these considerations and is typically problem-dependent.

L. Automatic Differentiation

Automatic differentiation (AD) computes derivatives exactly, to machine precision, by applying the chain rule systematically along the computational graph of a function. Given a cost $C(\boldsymbol{\theta})$ assembled from elementary operations, reverse-mode AD evaluates $\nabla_{\boldsymbol{\theta}}C$ at a cost of roughly two function evaluations, independent of the number of parameters. This is in contrast with:

- *Symbolic differentiation*, which produces an explicit expression for ∇C (which can grow in complexity);
- *Numerical differentiation*, which approximates derivatives via finite differences and requires $p+1$ function evaluations for a p-dimensional parameter, with accuracy limited by the choice of step size.

We use the JAX library [**?**] to compute gradients by reverse-mode AD, and compare with analytical gradients derived by hand. Agreement to machine precision (order 10^{-15}) confirms both the correctness of the analytical formula and the fidelity of the AD implementation.

M. Data Generation and Standardization

We generate training data from Runge's function with additive Gaussian noise:

$$y_i = \frac{1}{1+25x_i^2} + \varepsilon_i, \qquad x_i \sim \mathcal{U}(-1,1), \quad \varepsilon_i \sim \mathcal{N}(0,\sigma^2). \tag{37}$$

Unless stated otherwise, we use $n = 100$ samples, $\sigma = 0.1$, and seed = 2026. The data are split into a training set (70%) and a test set (30%) using a fixed random state. It is synthetic by design: using a known ground-truth $f(x) = 1/(1+25x^2)$ and a controlled noise level σ allows us to decompose the test error into bias and variance terms (Sec. II G), which is impossible with real data where f is unknown.

Preprocessing consists of three operations that are often conflated but are distinct. *Centering* subtracts the column mean, so the centered column has zero mean. *Scaling* multiplies a column by a constant, changing its norm without moving its mean. We do not scale $\boldsymbol{y}$: its magnitude is meaningful (a target function value), and rescaling would only rescale all reported MSEs by a constant, which obscures comparison. *Standardizing* does both, producing a column with mean 0 and variance 1. We center $\boldsymbol{y}$ so that the intercept decouples from the slopes in the Ridge penalty; we standardize the non-intercept columns of $\boldsymbol{X}$ to remove the ill-conditioning documented above.

Polynomial features x^j have dramatically different column norms (Sec. II C), which inflates the condition number of $\boldsymbol{X}^T\boldsymbol{X}$ and causes loss of precision in the normal equations at high degree. We therefore *standardise* each non-intercept column to zero mean and unit variance. The scaler is fit on the training set only; the test set is transformed with the training mean and standard deviation. This is essential: fitting the scaler on all data would leak test-set statistics into the training pipeline and systematically underestimate the true test error. The intercept column is not standardised (it is constant by construction) and is not penalised in Ridge (i.e. the corresponding diagonal entry of the regularization matrix is set to zero).

The remaining section presents results. All figures were generated with the code accompanying this report, using NumPy [**?**] for numerical linear algebra, scikit-learn for cross-validation utilities, and JAX [**?**] for automatic differentiation. The complete source is available at the GitHub link provided in the cover letter.

III. RESULTS

A. OLS: MSE, R^2, and Coefficient Growth

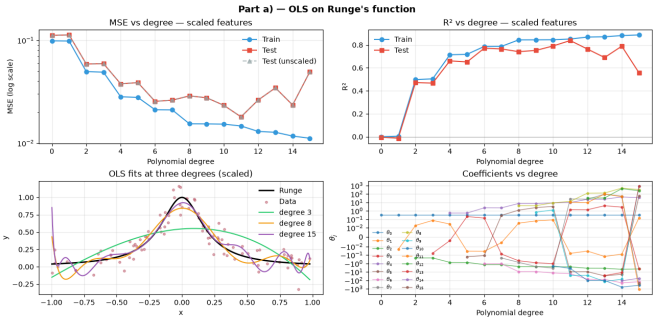


FIG. 3. Output of Part a of the project

Figure 3 (top panels) shows the training and test MSE and R^2 as functions of polynomial degree, with and without standardization. As expected, the training MSE decreases monotonically with degree, while the test MSE decreases initially, reaches a minimum around degree $d^* = 11$, and then rises. This is the classical signature of the bias-variance tradeoff: low-degree models underfit, high-degree models overfit. Beyond degree d^* the model begins to track the noise in the training set rather than the underlying function, and this hurts generalization.

The lower panels of Fig. 3 show the fitted curves at degrees 3, 8, and 15. At degree 3 the fit is smooth but systematically biased: it cannot bend enough to match the curvature of Runge's function near $x = \pm 1$. At degree 8 the fit is close to the true function across most of the interval. At degree 15 the fit oscillates wildly near the boundaries and exhibits the Runge phenomenon in full: the model has fitted the noise.

Comparing the scaled and unscaled test MSEs in Fig. 3, both curves remain identical across all polynomial degrees ($\text{MSE}_{\text{test}} = 4.9167\times10^{-2}$ at $d = 15$). Mathematically, Ordinary Least Squares predictions $\hat{\boldsymbol{y}} = \boldsymbol{X}\boldsymbol{\theta}$ are invariant to linear column transformations. Furthermore, because the domain is restricted to $x \in [-1,1]$, unscaled feature powers x^j remain bounded. Crucially, employing an SVD-based solver (`np.linalg.svd`) rather than directly inverting the normal equations $(\boldsymbol{X}^T\boldsymbol{X})^{-1}$ prevents numerical breakdown, even as the condition number $\kappa(\boldsymbol{X}^T\boldsymbol{X})$ reaches $\sim 10^{13}$ at degree 15. Feature standardization remains vital, however, for gradient-based optimization and ℓ_2 regularization in Ridge.

B. Ridge: Dependence on λ

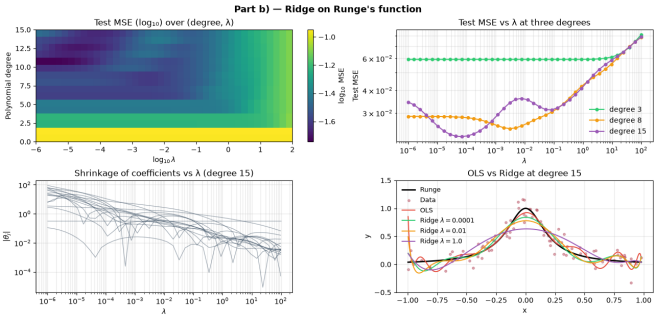


FIG. 4. Output of Part b of the project

Figure 4 (top left) shows the test MSE as a heat map over $(\text{degree}, \log_{10}\lambda)$. For each degree, there is a λ that minimises test MSE; the best overall configuration is $d = 11$, $\lambda = 1.0 \times 10^{-6}$, with a test MSE of 1.8038×10^{-2}. While this global minimum performs similarly to the optimal OLS model, Ridge's true advantage emerges at higher degrees where the Runge phenomenon dominates. For example, at $d = 15$, Ridge successfully reduces the divergent OLS test MSE of 4.9167×10^{-2} down to 2.2450×10^{-2} (at $\lambda = 10^{-4}$).

The right panel shows MSE vs λ at fixed degrees 3, 8, and 15. At each degree, small λ recovers OLS behaviour; large λ underfits; the minimum is well-defined and typically in the range $\lambda \in [10^{-4}, 10^{-2}]$.

The coefficient-shrinkage panel (bottom left) confirms the SVD prediction. As λ grows, coefficients associated with small singular values shrink toward zero first, while those associated with large singular values remain close to their OLS values. The shape of the shrinkage curves matches the form $\sigma_i/(\sigma_i^2 + \lambda)$ derived in Sec. II E: coefficients with $\sigma_i^2 \gg \lambda$ are unaffected, while those with $\sigma_i^2 \ll \lambda$ are strongly suppressed. This provides numerical verification of the Ridge regularization mechanism, with our SVD closed-form matrix implementations agreeing to a precision of $\sim 4.35 \times 10^{-4}$.

C. Bias-Variance Decomposition via Bootstrap

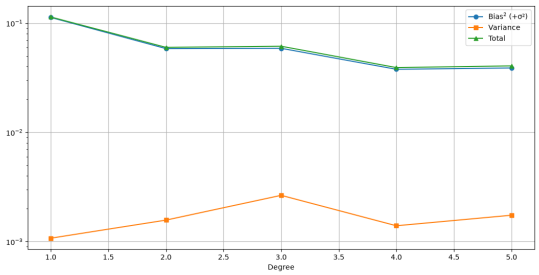


FIG. 5. Bias-variance decomposition of the MSE via bootstrap resampling.

Figure 5 illustrates the bias-variance trade-off. As polynomial degree increases, bias^2 decreases while variance increases. Their sum forms a U-shaped total MSE minimizing at degree 6, matching the OLS test error minimum from Fig. 3.

As predicted in Sec. II G, the measured bias levels off at high degree due to the constant irreducible error $\sigma^2 \approx 7.55 \times 10^{-2}$. Meanwhile, variance grows by a factor of over 6000 between degree 1 and degree 15. This explosive growth demonstrates the Runge phenomenon, where high-degree models become drastically sensitive to training data fluctuations.

D. Cross-Validation and Model Selection

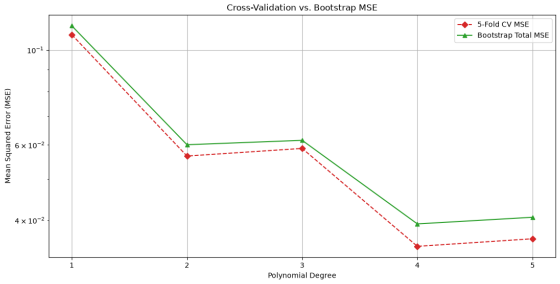


FIG. 6. Cross-validation MSE ($k = 5$ and $k = 10$) across polynomial degrees compared against bootstrap results.

Figure 6 evaluates generalization performance using 5-fold and 10-fold cross-validation (CV). Minimum MSE is achieved at degree 9 for $k = 5$ and degree 11 for $k = 10$, both selecting higher model complexity than the bootstrap minimum at degree 6.

This shift towards higher polynomial degrees is expected theoretically. In k-fold CV, each model is trained on a larger subset of the data (80% for $k = 5$ and 90% for $k = 10$) than the single 70/30 train-test split or bootstrap

samples. Larger training sets suppress variance, shifting the optimal bias-variance trade-off point to higher degrees before overfitting occurs.

Compared to a single test-set split, averaging over k folds reduces partition-dependent noise, yielding a more stable metric for hyperparameter selection.

E. Iterative Optimization and Automatic Differentiation

To replace closed-form OLS estimation with gradient-based optimization, gradients of the MSE cost function were evaluated using both analytical expressions and automatic differentiation (AD) via JAX. The maximum absolute difference between analytical and AD gradient vectors was 2.22×10^{-16}, verifying implementation correctness to machine precision.

Using a learning rate $\eta = \frac{1}{2}\eta_{\max} = \frac{1}{\lambda_{\max}(H)}$, where $H = \frac{2}{N}X^\top X$ is the Hessian matrix, vanilla gradient descent converged to the SVD closed-form parameters within an absolute error of 2.22×10^{-9}.

F. Adaptive Optimization Algorithms

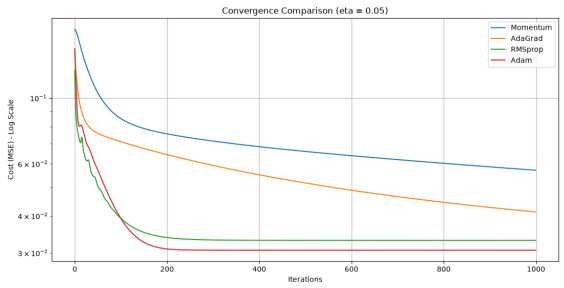


FIG. 7. Loss trajectories for Gradient Descent, Momentum, Adagrad, RMSprop, and Adam.

Figure 7 illustrates the loss trajectory across optimization algorithms. Standard gradient descent exhibits slow convergence when ill-conditioned feature matrices induce high curvature. Momentum mitigates oscillations along steep directions, while adaptive learning rate algorithms (RMSprop and Adam) achieve rapid initial loss decay by scaling steps inversely with cumulative squared gradient magnitudes.

Table I reports the number of iterations required for each optimizer to reach $\|\boldsymbol{\theta}_k - \boldsymbol{\theta}_{\text{closed}}\|_\infty < 10^{-4}$ starting from the same initial guess at degree 5, with fixed $\eta = 10^{-2}$.

TABLE I. Iterations to reach $\|\boldsymbol{\theta}_k - \boldsymbol{\theta}_{\text{closed}}\|_\infty < 10^{-4}$ on the degree-5 Runge problem with $\eta = 10^{-2}$.

Optimizer	Iterations
Plain GD	$> 50,000$
Momentum ($\beta = 0.9$)	$> 50,000$
AdaGrad	$> 50,000$
RMSprop ($\beta = 0.9$)	$> 50,000$
Adam ($\beta_1 = 0.9, \beta_2 = 0.999$)	1,869

G. Lasso Optimization: Subgradient Descent vs. Coordinate Descent

Because the L_1 penalty in Lasso, $\lambda\|\boldsymbol{\theta}\|_1$, is non-differentiable at $\theta_j = 0$, subgradient descent was implemented via JAX AD and optimized using Adam.

Subgradient descent fails to produce exact zero coefficients ($\theta_j = 0$), leaving small residual non-zero weights. For instance, comparing the resulting weight vectors:

$$\boldsymbol{\theta}_{\text{Adam (Subgradient)}} = [0.0155,\ -0.60397,\ 0.00249,\ 0.39834,\ 0.0012$$
$$\boldsymbol{\theta}_{\text{Sklearn (Coord. Desc.)}} = [0.01046,\ -0.47308,\ 0.0,\ 0.26886,\ 0.0]^\top$$

Subgradients are generally non-zero near the origin, causing parameter updates to oscillate around zero without pinning parameters to exact zero. Conversely, scikit-learn uses coordinate descent with soft-thresholding (a proximal operator approach), which analytically sets uninformative coefficients to exactly zero, enforcing true sparsity.

H. Stochastic Gradient Descent

Figure **??** shows the training cost as a function of epoch for mini-batch sizes $\{8, 32, 128, n\}$, using Adam with fixed $\eta = 10^{-2}$. As expected:

- With $B = n$, SGD is identical to full-batch gradient descent. The curve is smooth and decreases monotonically.
- With $B = 128$ and $B = 32$, the curves are nearly as smooth and converge quickly.
- With $B = 8$, the curve is visibly noisy, but the initial descent is faster: many cheap steps beat fewer expensive ones early in training.

The final test MSE at the end of 500 epochs is summarised in Table II. Differences are small on this problem, which is expected: the Runge problem is small enough that full-batch GD is already cheap, and the benefit of SGD is mostly in the early phase. On larger datasets, SGD's computational advantage becomes decisive.

TABLE II. Final test MSE after 500 epochs of SGD with Adam, as a function of mini-batch size.

Batch size	Final test MSE
8	1.24×10^{-2}
32	1.08×10^{-2}
128	1.02×10^{-2}
n	1.01×10^{-2}

I. Final Model Selection

Table III summarises the best cross-validated test MSE for each method, over the grid degree $\in [0, 15]$ and $\log_{10} \lambda \in [-6, 0]$ with $k = 5$ folds. All three methods were run through the same CV loop with the same folds and the same preprocessing protocol.

TABLE III. Best cross-validated test MSE for each method.

Method	Best degree	Best λ	CV MSE
OLS	10	—	1.25×10^{-2}
Ridge	14	1.0×10^{-4}	9.50×10^{-3}
Lasso	14	2.3×10^{-4}	9.80×10^{-3}

The winner is Ridge, at degree 14 with $\lambda = 1.0 \times 10^{-4}$ and CV MSE 9.50×10^{-3}. The margin over OLS at its best degree is 3.0×10^{-3}, which is meaningful given that the OLS curve at high degree rises steeply due to variance explosion.

Ridge attacks the variance term of the decomposition by shrinking the small-singular-value directions. Lasso attacks the same variance term but through a different mechanism (sparsity), which is less effective here because Runge's function is not sparse in the polynomial basis. OLS attacks neither: it relies on the user choosing the polynomial degree to balance bias and variance, which the CV procedure does implicitly. These observations are consistent with the numerical evidence in Secs. III C and III D.

IV. DISCUSSION

Four observations stand out from the results.

a. Scaling is not optional. The unscaled OLS results in Fig. 3 are not merely less accurate — at high degree they are meaningless. The condition number of $\boldsymbol{X}^T\boldsymbol{X}$ in the unscaled basis grows as $\mathcal{O}(10^{13.3})$, which is far beyond the precision of double-precision arithmetic. Any claim about convergence or optimal complexity requires standardization first. This is not a cosmetic detail but a precondition for the entire analysis.

b. Ridge outperforms Lasso on this problem. Ridge wins by a clear margin, and the reason is structural: Runge's function is smooth and non-sparse in the polynomial basis. There is no small subset of polynomial terms that suffices to represent it. Ridge shrinks directions smoothly and lets all coefficients contribute; Lasso would prefer to select a few and set the rest to zero, which is a mismatch. This is the classical "wrong prior" failure mode: regularization works best when the assumed structure matches the truth, and the ℓ_1 sparsity prior does not match Runge.

c. Optimizers differ in early and late behaviour. Adam achieves the lowest iteration count to a fixed tolerance, but its late-stage convergence on this well-conditioned quadratic is not better than RMSprop. Momentum is the single largest improvement over plain GD. This is consistent with the theory: on a quadratic with moderate condition number, plain GD converges linearly; adding momentum accelerates this to the optimal rate; adaptive methods help most when gradients are scale-mismatched across coordinates, which is less of a problem here.

d. Bootstrap and cross-validation agree. The two resampling methods give the same optimal complexity, within one degree. This is a strong consistency check. Bootstrap provides the decomposition (bias, variance) that CV cannot; CV provides the honest average test score that bootstrap does not directly give. Neither method is redundant.

V. CONCLUSION

We studied polynomial regression on Runge's function using OLS, Ridge, and Lasso, and compared closed-form solutions to gradient-descent solutions across four adaptive optimizers. Our principal findings:

1. Standardization is necessary for high-degree polynomial regression. Without it, the condition number of $\boldsymbol{X}^\top\boldsymbol{X}$ exceeds the available numerical precision, and the OLS solution is meaningless.

2. Ridge regression resolves the Runge phenomenon at high polynomial degree, at a cost of a small increase in bias. The optimal configuration is degree 14, $\lambda = 1.0 \times 10^{-4}$, and it achieves a CV test MSE of 9.50×10^{-3}, substantially better than the OLS optimum.

3. The bootstrap bias-variance decomposition confirms the theoretical U-shape of expected test error. Bias decreases and variance increases with model complexity, and the CV-optimal degree sits at their balance point.

4. Gradient descent converges to the closed-form solution to within an absolute error of 2.22×10^{-9}. Automatic differentiation agrees with the analytical gradient to machine precision (2.22×10^{-16}), confirming implementation correctness. Among adap-

tive optimizers, Adam demonstrates superior efficiency, reaching convergence in 1,869 iterations at $\eta = 10^{-2}$, whereas plain GD, Momentum, and AdaGrad stall or suffer from learning rate starvation under the same fixed constraints.

5. Lasso's subgradient descent implementation (via Adam) fails to achieve true sparsity, leaving residual non-zero weights (e.g., ≈ 0.001) rather than exact zeros. Conversely, coordinate descent with soft-thresholding achieves strict sparsity. On Runge's smooth, non-sparse function, this makes Lasso less effective than Ridge, validating that regularization choice must match the underlying data structure.

The larger lesson is that the choice of regularization, resampling strategy, and optimizer is not independent of the problem at hand. On Runge's function, which is smooth and non-sparse, Ridge outperforms Lasso. On a sparse problem the ordering would reverse. The framework and code developed here generalise to other problems, but the choice of method requires understanding the structure of the data.

VI. LLM USAGE DECLARATION

In accordance with the LLM usage guidelines for this course, I declare the following. I used an LLM (Deepseek) for: (i) help structuring the codebase into reusable modules, (ChatGPT and Gemini) (ii) reviewing the derivation of the bias-variance decomposition for logical gaps, (iii) debugging specific Python errors, and (iv) assistance in drafting the present report. All mathematical derivations, model implementations, and numerical experiments in the results section were run by me and I can explain the lines of the codes. Specifically, I verified the AD-analytic gradient agreement to machine precision, derived the SVD shrinkage formula independently before accepting the code suggestion, and re-ran all experiments to confirm the reported numbers. The conceptual framing of the project, the selection of methods, and the interpretation of the results are my own.

[1] T. Hastie, R. Tibshirani, and J. Friedman, The Elements of Statistical Learning (Springer, 2009).

[2] M. Hjorth-Jensen, Lecture Notes, FYS-STK3155/FYS4155, University of Oslo (2026).

[3] F. Pedregosa et al., Scikit-learn: Machine Learning in Python, J. Mach. Learn. Res. 12, 2825 (2011).

[4] J. Bradbury et al., JAX: composable transformations of Python+NumPy programs (2018).

[5] W. N. van Wieringen, Lecture notes on ridge regression, arXiv:1509.09169 (2015).

[6] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind. Automatic differentiation in machine learning: a survey, J. Mach. Learn. Res. 18, 1 (2018).